

Telegram-Stremio Feature Roadmap & Improvement Ideas

This document outlines potential features, fixes, and improvements for the Telegram-Stremio bot, organized by category and priority.

High Priority (High Impact, Critical Fixes)

1. Video Caching System ★ BIGGEST BANDWIDTH SAVER

Problem: Every stream downloads from Telegram in real-time, using 2x bandwidth (download + upload)

Solution: Implement local caching

- Cache frequently watched videos on disk
- LRU (Least Recently Used) eviction when disk is full
- Configurable cache size limit (e.g., 50GB, 100GB)
- Cache warm-up for popular content
- **Impact:** Can reduce bandwidth by 50-80% for popular content

Implementation:

```
# Backend/helper/video_cache.py
class VideoCache:
    - check_cache(file_id) -> bool
    - get_cached_path(file_id) -> str
    - cache_video(file_id, stream) -> None
    - evict_lru() -> None
    - get_cache_stats() -> dict
```

2. Automatic Channel Access Verification

Problem: CHANNEL_INVALID errors when userbot loses access to channels

Solution:

- Periodic channel access check (every 6 hours)
- Auto-notification to admin when access is lost
- Dashboard indicator showing channel health status
- Auto-remove inaccessible channels from search list (with notification)

Files to modify:

- Backend/helper/channel_verifier.py (new)
- Add to dashboard: channel status indicators

3. Enhanced Error Handling & Retry Logic

Problem: Streams fail silently when Telegram is temporarily unavailable

Solution:

- Exponential backoff retry for failed Telegram API calls

- Circuit breaker pattern for consistently failing channels
- Better error messages in Stremio (not just generic 500)
- Fallback to alternate clients when one fails

Files to modify:

- Backend/helper/custom_dl.py
- Backend/fastapi/routes/stream_routes.py

4. Rate Limiting & DDoS Protection

Problem: No protection against abuse or bandwidth exhaustion

Solution:

- Per-token rate limiting (max streams/hour, max GB/day)
- IP-based rate limiting for public instances
- Configurable limits in admin panel
- Auto-ban suspicious patterns

Implementation:

```
# Backend/helper/rate_limiter.py
- check_rate_limit(token, ip) -> bool
- track_request(token, ip, bytes_consumed)
- get_user_usage(token) -> dict
```



Medium Priority (Great UX Improvements)

5. Resume Watching / Progress Tracking

Problem: Users lose their position when they stop watching

Solution:

- Track watch progress per user per video
- Store in MongoDB: {user_id, video_id, position, last_watched}
- Resume prompt in Stremio (via manifest update)
- Web dashboard showing "Continue Watching"

Database Schema:

```
watch_progress: {
  user_id: int,
  imdb_id: string,
  season: int,
  episode: int,
  position_seconds: int,
  duration_seconds: int,
  last_watched: datetime
}
```

6. Watch History & Statistics

Solution:

- Per-user watch history in dashboard

- Most watched content
- Viewing trends graph
- Export history as CSV

7. Favorites / Watchlist

Solution:

- Users can favorite movies/shows
- Separate catalog in Stremio: “My Favorites”
- Add/remove via web dashboard or bot command

8. Advanced Search & Filters

Solution:

- Filter by: genre, year, quality, language
- Sort by: date added, popularity, rating
- Search suggestions as you type
- Recent searches history

9. Subtitle Support

Problem: No subtitle handling

Solution:

- Auto-detect .srt files in Telegram channels
- Link subtitles to videos by filename matching
- Serve subtitles via `/subtitle/{file_id}.srt` endpoint
- Multiple language support

10. Quality Selection

Problem: Users can't choose quality if multiple versions exist

Solution:

- When multiple qualities exist (720p, 1080p, 4K), show all in Stremio
- Format: 🎬 1080p WEB-DL [5.2 GB]
- Let user pick their preferred quality



Analytics & Monitoring

11. Comprehensive Analytics Dashboard

Metrics to track:

- Real-time active streams
- Bandwidth usage per day/week/month
- Most popular content
- Peak usage times
- User growth over time
- Average video quality delivered
- Error rates by type

Visualization:

- Line charts for trends

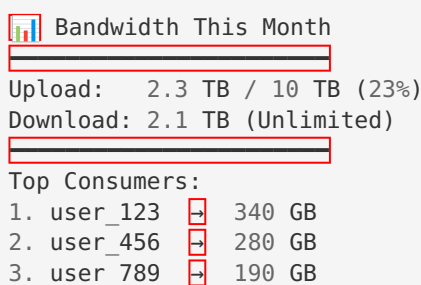
- Pie charts for content distribution
- Heatmap for usage patterns

12. Bandwidth Usage Tracking

Solution:

- Per-user bandwidth consumption
- Per-video bandwidth consumption
- Predictions for month-end usage
- Alerts when approaching Oracle 10TB limit
- Export reports for capacity planning

Dashboard additions:



13. Health Monitoring System

Solution:

- `/health` endpoint for uptime monitoring
- Automatic alerts (Telegram message to owner) when:
 - Database connection fails
 - Disk space < 10%
 - Memory usage > 90%
 - Any client fails repeatedly
- Prometheus metrics export (optional)



Security Enhancements

14. Enhanced Token Security

Improvements:

- Token expiration with auto-renewal
- Token rotation on suspicious activity
- Device fingerprinting to detect token sharing
- Max concurrent streams per token
- IP whitelist per token (optional)

15. Audit Logging

Solution:

- Log all admin actions (plan changes, token revokes, settings updates)
- Log authentication attempts
- Store in separate `audit_logs` collection

- View logs in dashboard
- Export logs as JSON

16. Two-Factor Authentication for Admin

Solution:

- OTP-based 2FA for web panel login
 - QR code setup with TOTP apps
 - Backup codes for recovery
-

Developer Experience & DevOps

17. Automated Backup System

Solution:

- Daily MongoDB backup to cloud (S3, Backblaze B2)
- Configurable retention (keep last 7 days)
- One-click restore from dashboard
- Backup verification

18. Configuration Validation

Solution:

- Validate `config.env` on startup
- Check MongoDB connectivity before starting
- Verify Telegram credentials
- Test channel access
- Fail fast with clear error messages

19. Better Logging

Improvements:

- Structured JSON logging (easier to parse)
- Log levels configurable via dashboard
- Separate log files: `access.log`, `error.log`, `admin.log`
- Log rotation (max 100MB per file)
- Real-time log viewer in dashboard

20. Docker Improvements

Enhancements:

- Multi-stage build (smaller image size)
 - Health check in Dockerfile
 - Non-root user execution
 - Volume for logs and cache
 - Docker secrets support
-

Content Management

21. Automatic Duplicate Detection

Problem: Same movie uploaded multiple times with different names

Solution:

- Compare IMDb IDs and file hashes
- Merge duplicates automatically
- Keep highest quality version
- Notify admin of duplicates found

22. Batch Operations

Solution:

- Bulk delete videos
- Bulk metadata update
- Bulk quality upgrade
- CSV import/export for content library

23. Content Categories

Solution:

- Tag content: Action, Comedy, Horror, etc.
- Auto-categorize using TMDb genres
- Separate catalogs per category in Stremio
- Filter by multiple categories

24. Multi-Language Support

Solution:

- Detect audio languages from filename
- Filter by language in search
- Show available languages in Stremio title
- Support for dual-audio files

Performance Optimizations

25. Database Query Optimization

Improvements:

- Add indexes on frequently queried fields
- Use MongoDB aggregation pipelines
- Cache frequent queries (Redis optional)
- Pagination for large results

Indexes to add:

```
db.movies.createIndex({imdb_id: 1})
db.movies.createIndex({title: "text"})
db.tv_shows.createIndex({imdb_id: 1, season: 1, episode: 1})
db.streams.createIndex({token: 1, timestamp: -1})
```

26. CDN Integration (Advanced)

Solution:

- Integrate with BunnyCDN or Cloudflare
- Cache video streams at edge locations
- Reduce load on Oracle VPS
- Better performance for global users

27. Streaming Optimizations

Improvements:

- Adaptive chunk size based on connection speed
 - Prefetch next chunks while streaming
 - Better range request handling
 - Reduce Time To First Byte (TTFB)
-



Bug Fixes & Code Quality

28. Unicode Filename Handling (Already Fixed)

- Fixed Content-Disposition header encoding
- Added RFC 6266 compliance

29. Better Error Recovery

Issues to fix:

- Handle expired Telegram session strings gracefully
- Auto-reconnect on MongoDB connection drops
- Recover from partial file transfers
- Handle corrupt video files

30. Code Refactoring

Improvements:

- Extract magic numbers to constants
 - Add type hints throughout
 - Improve code documentation
 - Unit tests for critical functions
 - Integration tests for API endpoints
-



Nice-to-Have Features

31. Mobile App (Long-term)

- Native Android/iOS app
- Push notifications for new content
- Offline download support
- Better mobile UX than web panel

32. Content Recommendations

- “You might also like” suggestions

- Based on watch history
- TMDb recommendations API

33. Multi-User Sharing

- Family accounts (sub-users under main account)
- Per-user watch history
- Parental controls

34. Torrent Integration (Advanced)

- Auto-download popular content from torrents
- Upload to Telegram automatically
- Automated library expansion

35. Webhook Notifications

- Discord webhook when new content added
- Telegram group notifications
- RSS feed for new content



Implementation Priority Guide

Start Here (Highest ROI):

1.  Unicode fix (DONE)
2.  Video caching system (massive bandwidth savings)
3.  Automatic channel verification
4.  Rate limiting & bandwidth tracking
5.  Better error handling

Next Phase:

1. Resume watching / progress tracking
2. Analytics dashboard
3. Watch history
4. Health monitoring
5. Backup system

Future Enhancements:

1. CDN integration
2. Advanced security features
3. Mobile app
4. AI-powered recommendations



Quick Wins (Easy to Implement)

Super Easy (< 2 hours):

- Add `/health` endpoint

- Configuration validation on startup
- Better log formatting
- Admin action audit log
- Channel status indicators in dashboard

Easy (< 1 day):

- Watch history tracking
- User bandwidth usage display
- Token expiration
- Basic rate limiting
- Favorites list

Contributing

Want to implement any of these features?

1. Fork the repo
2. Create a feature branch
3. Implement with tests
4. Submit a PR with documentation

Feedback

Have more ideas? Open an issue on GitHub or discuss in the community!